Atelier 7 : GitLab CI/CD + SonarQube Cloud

Rapport de réalisation · Omaima El Harti · Novembre 2026

1. Objectif

Mettre en place un pipeline CI/CD sur GitLab qui analyse automatiquement la qualité et la sécurité d'un mini-site (HTML, CSS, JavaScript) avec SonarQube Cloud, puis déploie le site sur GitLab Pages uniquement si le Quality Gate est respecté.

L'atelier repose sur deux versions du même projet :

- **V2** : code volontairement vulnérable (injection XSS), pour observer le blocage du déploiement.
- **V1** : code correct, pour observer l'analyse réussie et le déploiement.

2. Architecture du pipeline

Le pipeline contient deux jobs :

1. **sonarqube-check** (stage `quality`) : lance `sonar-scanner`, envoie l'analyse à SonarQube Cloud et attend le résultat du Quality Gate (`sonar.qualitygate.wait=true`).
2. **deploy-pages** (stage `deploy`) : copie `index.html`, `style.css` et `script.js` dans `public/` puis publie sur GitLab Pages.

Le job de déploiement ne s'exécute que si le job de qualité réussit.

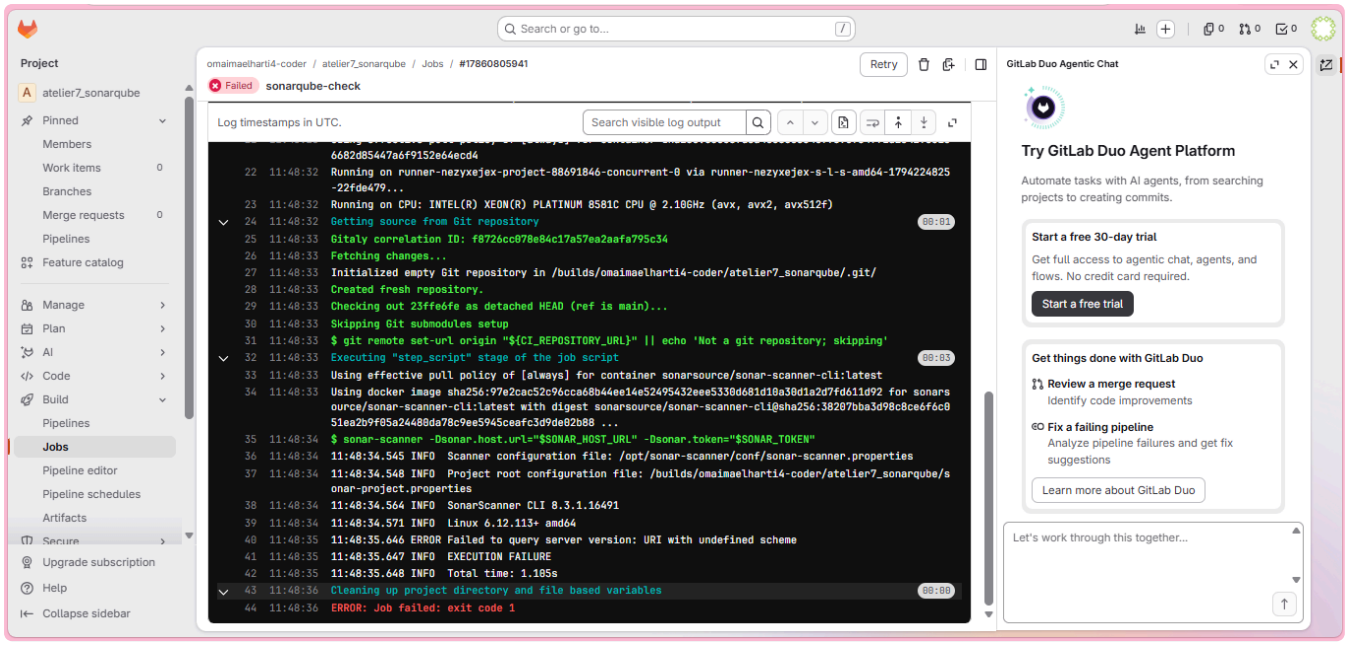
3. Mise en place

1. Création du projet GitLab et clonage en local.
2. Création des fichiers `index.html`, `style.css`, `script.js`.
3. Création du projet sur SonarQube Cloud (connexion avec GitLab) et récupération de la *Project Key* et de l'*Organization Key*.
4. Création du fichier `sonar-project.properties`.
5. Création d'un token SonarQube Cloud.
6. Ajout des variables CI/CD GitLab : `SONAR_TOKEN` et `SONAR_HOST_URL` (`https://sonarcloud.io`).
7. Création du fichier `.gitlab-ci.yml` avec les deux jobs.

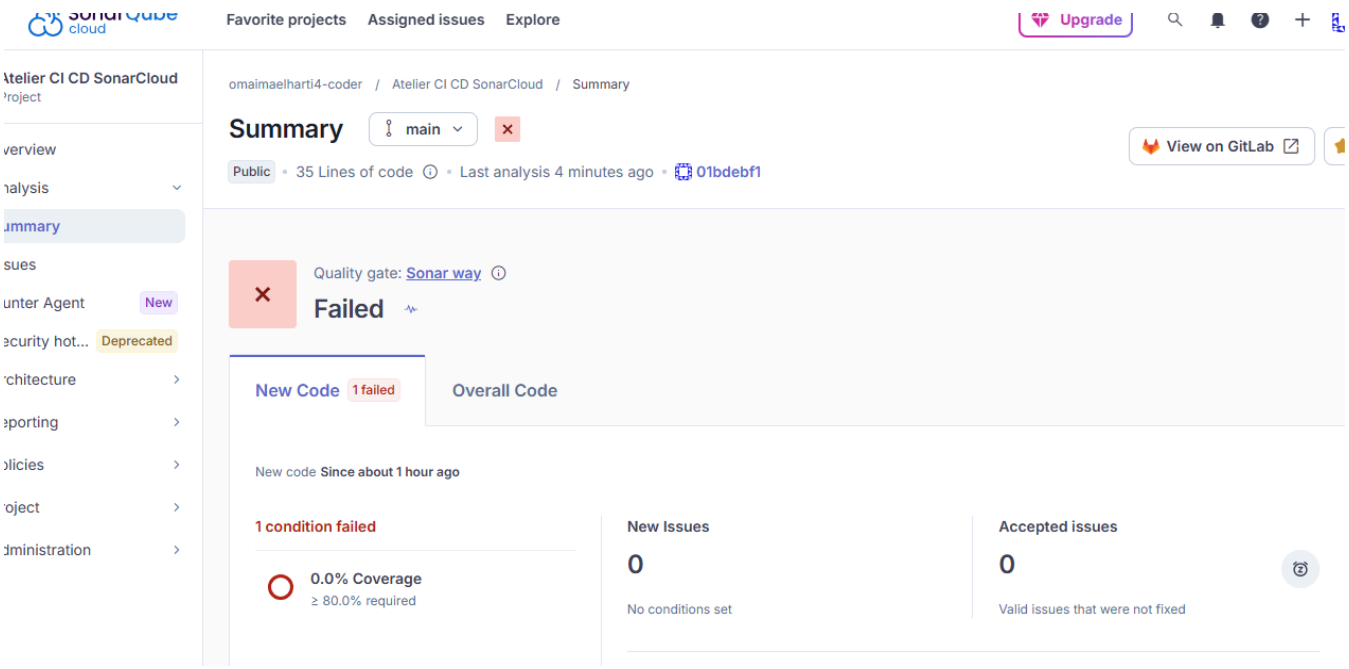
4. Quand ça ne marchait pas : problèmes rencontrés et solutions

Problème rencontré	Cause	Solution
`git add .` ajoutait tout le dossier utilisateur ; `No configured push destination`	Un dépôt `.git` existait par erreur dans `C:\Users\HP`	Renommage du `.git` erroné, puis clonage du projet dans un dossier propre
`git push` rejeté (`fetch first`)	Le dépôt distant contenait des commits absents en local	`git pull --rebase` puis nouveau push
Impossible d'importer le groupe GitLab dans SonarQube (token invalide / permissions insuffisantes)	Token de type *fine-grained* au lieu d'un token avec le scope `api`	Utilisation d'un token adapté et import du projet personnel
Pipeline en échec sans aucun job	GitLab demandait la vérification du compte avant d'exécuter des pipelines	Vérification du compte
`Failed to query server version: URI with undefined scheme`	La variable `SONAR_HOST_URL` n'était pas définie dans le projet	Ajout de `SONAR_HOST_URL` et `SONAR_TOKEN` dans Settings → CI/CD → Variables
`QUALITY GATE STATUS: FAILED` alors que le code ne contenait aucune issue	Condition *Coverage ⩾ 80 %* du Quality Gate « Sonar way », alors qu'il n'y a aucun test dans l'atelier	Ajout de `sonar.coverage.exclusions=**/*` dans `sonar-project.properties` (un Quality Gate personnalisé n'est pas disponible dans le plan gratuit)

À noter aussi : l'erreur `You are running CI analysis while Automatic Analysis is enabled` se résout en désactivant *Automatic Analysis* dans Administration → Analysis method.



Log du job `sonarqube-check` : erreur `URI with undefined scheme` (variable `SONAR_HOST_URL` absente).



SonarQube Cloud : 0 nouvelle issue, mais Quality Gate en échec à cause de la condition Coverage (0,0 % pour 80 % requis).

5. Cas d'échec : V2 (code vulnérable)

Code volontairement vulnérable (`script.js`) :

```
const form = document.getElementById("demo-form");
const input = document.getElementById("user-input");
const message = document.getElementById("message");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  message.innerHTML = input.value;
});
```

L'utilisation de `innerHTML` avec une saisie non fiable permet une injection XSS. Exemple de saisie qui exécute du code :

```
<img src=x
onerror="document.body.insertAdjacentHTML('beforeend','<h2>Injection</h2>')">
```

Résultat attendu après `git push` :

- SonarQube Cloud détecte l'issue « DOM XSS via unsanitized user input » (sévérité Blocker).
- Le Quality Gate passe en **Failed**.
- Le job `sonarqube-check` échoue et le job `deploy-pages` n'est pas exécuté : le code vulnérable n'est pas déployé.

(Insérer ici les captures d'écran : pipeline en échec, dashboard SonarQube, page Issues.)

6. Cas de réussite : V1 (code correct)

Code correct (`script.js`) :

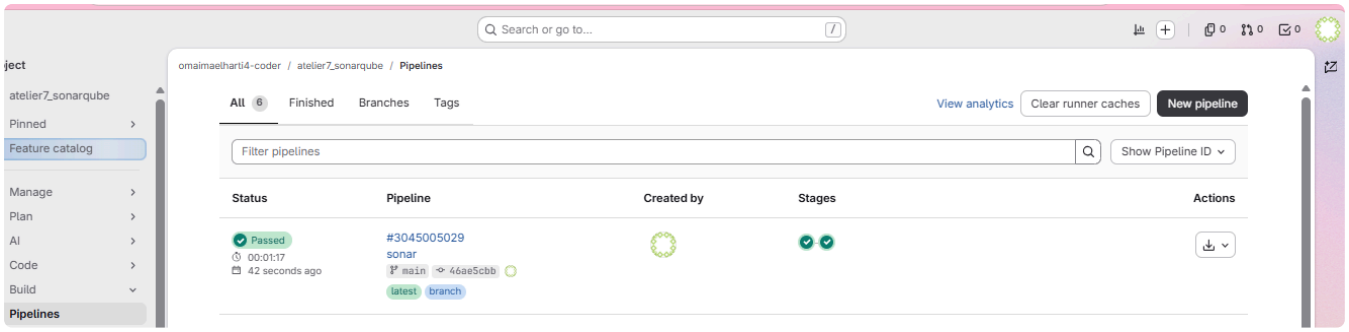
```
const bouton = document.getElementById("btn");
const message = document.getElementById("message");

bouton.addEventListener("click", function () {
  message.textContent = "Le JavaScript fonctionne correctement !";
});
```

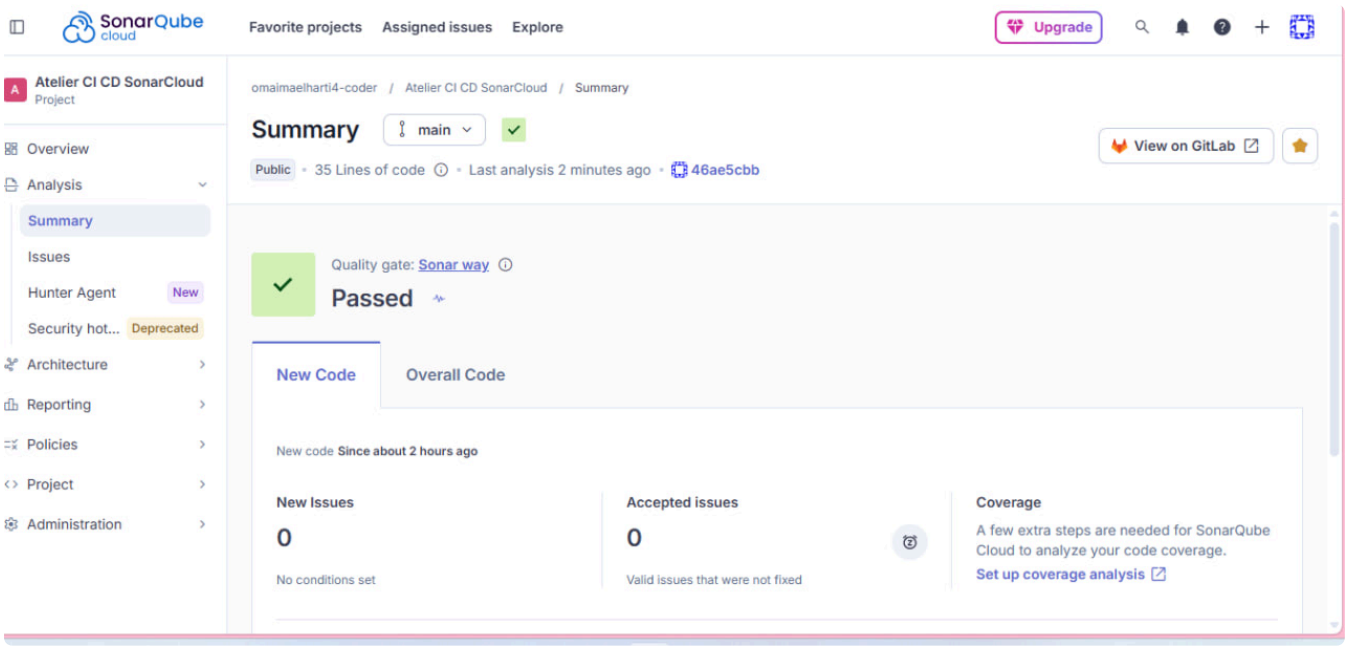
L'utilisation de `textContent` n'interprète pas le contenu comme du HTML : aucune faille XSS.

Résultat obtenu :

- Analyse SonarQube Cloud : 0 issue, Quality Gate **Passed**.
- Pipeline GitLab : les deux jobs `sonarqube-check` et `deploy-pages` réussissent.
- Le site est publié sur GitLab Pages.



Pipeline GitLab : statut Passed, les deux stages (quality et deploy) sont validés.



SonarQube Cloud : Quality Gate Passed, 0 nouvelle issue sur la branche main.

7. Conclusion

Après chaque `git push`, GitLab lance le pipeline, SonarQube analyse le code et évalue le Quality Gate, et le déploiement n'a lieu que si la qualité est conforme. Cet atelier montre que GitLab CI/CD automatise les étapes, que SonarQube contrôle la qualité et la sécurité du code, et que le Quality Gate peut empêcher le déploiement d'un code qui ne respecte pas les critères définis.

Compétences mobilisées : Git, GitLab CI/CD, SonarQube Cloud, GitLab Pages, analyse de sécurité (XSS), débogage de pipeline.